

Open-Source SOC Pipeline

Section 1: Project Overview and Objectives

- **Project Goal:** To engineer, deploy, and harden an end-to-end, open-source Security Operations Center (SOC) pipeline.
- **Problem Being Solved:** Establishing a functional, automated threat detection, orchestration, and case management lab that can run reliably within the strict hardware constraints of a single 16 GB RAM host laptop without severely degrading its physical Solid State Drive (SSD).
- **Architecture Flow:** The pipeline relies on a decoupled, event-driven architecture where endpoint telemetry (sysmon logs) streams to the SIEM, alerts pass through a translation/validation loop via webhooks, trigger the SOAR engine for enrichment, and are automatically pushed to a Case Management platform for analyst remediation.
- **Key Requirements:**
 - Must map detections to MITRE ATT&CK techniques.
 - Must dynamically apply Service Level Agreements (SLAs) based on alert severities.
 - Must utilize cloud-based threat intelligence APIs rather than locally hosted databases to preserve memory.
- **SLA Determination Matrix:**
 - **Critical:** Time to Acknowledge (TTA) $\leq$ 15 Minutes; Time to Contain (TTC) $\leq$ 1 Hour; Time to Resolve (TTR) $\leq$ 4 Hours. Example: Credential Dumping (T1003.001).
 - **High:** TTA $\leq$ 30 Minutes; TTC $\leq$ 4 Hours; TTR $\leq$ 24 Hours. Example: Event Log Clearing (T1070.004), Ransomware.
 - **Medium:** TTA $\leq$ 2 Hours; TTC $\leq$ 12 Hours; TTR $\leq$ 48 Hours. Example: Unauthorized Scheduled Task Creation (T1053.005).
 - **Low:** TTA $\leq$ 8 Hours; TTC: N/A; TTR $\leq$ 7 Days. Example: Policy violations, basic enumeration.

Section 2: Architecture and Environment Layout

- **Host Machine:** 16 GB RAM total. 4 GB RAM allocated to the Host OS to run VMware and browsers.
- **SOC-Backend (Ubuntu Server VM):**
 - **OS:** Ubuntu Server (64-bit, 22.04 LTS / 24.04 LTS), CLI only (headless).
 - **Compute:** 8 GB RAM, 2 vCPUs.
 - **Storage:** 40 GB – 50 GB Thin Provisioned disk (Note: previously 38GB, which caused exhaustion; 60 GB - 80 GB is the current recommended requirement).
 - **Networking:** Static IP [192.168.196.132](#) bound via Netplan to interface [ens33](#). Docker Bridge Network [soc-network](#) configured for container communication.
 - **Purpose:** Hosts the native Wazuh installation together with the Dockerized SOC services. (SIEM, SOAR, Case Management).
- **Endpoint-Victim (Windows 10/11 VM):**
 - **OS:** Windows 10 or 11 Enterprise/Evaluation.
 - **Compute:** 4 GB RAM, 2 vCPUs.
 - **Storage:** 50 GB Thin Provisioned disk.
 - **Networking:** DHCP IP [192.168.196.133](#).
 - **Purpose:** Generates telemetry via Sysmon and acts as the target for Atomic Red Team adversary emulation.

Section 3: Technology Stack

- **Hypervisor:** VMware Workstation. Chosen for virtualization.
- **SIEM: Wazuh** (Version 4.7.2). Chosen for log ingestion, decoding, and rule matching. Replaced Splunk in the initial plan.
- **SOAR: Shuffle** (latest ghcr.io images). Orchestrates webhook data, transforms JSON, and interacts with external APIs.
- **Case Management: TheHive** (Version 5.2). Tracks incident workflows. Runs on port 9000.

- **Databases:**
 - **OpenSearch** (Version 2.11.0): Backend for both Wazuh (Wazuh Indexer) and Shuffle (shuffle-opensearch).
 - **Cassandra** (Version 4.0): Dedicated backend database for TheHive.
- **Endpoint Telemetry: Sysmon** (System Monitor) with SwiftOnSecurity configuration. Captures granular process creations, file mods, and network connections.
- **Threat Intel (Cloud): AlienVault OTX** (Primary), VirusTotal, AbuseIPDB. Used via Shuffle APIs for IOC enrichment.
- **Adversary Emulation: Atomic Red Team (ART)**. Powershell-based framework to safely execute mapped MITRE ATT&CK techniques.
- **Dropped Technologies:**
 - **MISP (Malware Information Sharing Platform)**: Dropped due to strict 16 GB laptop constraints. MISP required 2-4 GB of RAM. Replaced by free cloud API nodes in Shuffle.
 - **Cortex**: Dropped to save RAM. TheHive throws warnings looking for Cortex, but the pipeline functions without it.

Section 4: Architectural Decisions and Tradeoffs

- **Thin vs. Thick Provisioning:** VM virtual disks must be Thin Provisioned. This prevents VMware from instantly carving out 100+ GB of physical space on the host SSD.
- **Physical SSD Wear Prevention (The "Red Line"):** Heavy swapping in a VM degrades laptop SSDs. Two major decisions mitigate this:
 1. The VMware `.vmx` file was edited to include `mainMem.useNamedFile = "FALSE"` to prevent writing VM RAM state to the physical drive.
 2. Ubuntu Guest OS `vm.swappiness` was set to `1` to force the kernel to exhaust all 8GB of physical RAM before utilizing the swap file.
- **OpenSearch Bootstrapping:** To save memory and bypass heavy TLS/SSL certificate generation containers, Shuffle's OpenSearch instance was configured with `plugins.security.disabled=true`. This allows TheHive to connect to it via plain HTTP.
- **TheHive Connector vs. HTTP Node:** The default TheHive App in Shuffle contains formatting bugs (e.g., sending unquoted placeholders like `${flag}` which crash TheHive's

JSON parser). The decision was made to bypass the app entirely and use a raw HTTP POST node in Shuffle to send perfectly formatted JSON directly to TheHive's API.

- **Alert Escalation:** The architecture explicitly avoids creating "Cases" automatically. Alerts are sent to TheHive's "Alerts" dashboard for human analyst triage; verified alerts are then promoted to Cases.

Section 5: Implementation and Configuration Details

1. Ubuntu OS Hardening & Limits

- **Virtual Memory Map:** OpenSearch requires `vm.max_map_count=262144` configured in `/etc/sysctl.conf` to prevent immediate container crashes.
- **Docker Log Caps:** `/etc/docker/daemon.json` is configured with `{"log-driver": "json-file", "log-opts": {"max-size": "10m", "max-file": "3"}}` to prevent infinite log expansion.
- **Automated Pruning:** A root cron job executes `/usr/local/bin/soc-cleanup.sh` at 2:00 AM daily, running `docker system prune -f`, clearing syslog, and truncating container logs.

2. Network Layout & Ports

- **Wazuh:** 1514 (UDP/TCP, Agent logs), 1515 (TCP, Registration), 55000 (TCP, REST API), 443 (TCP, Dashboard web UI), 9200 (TCP, Indexer).
- **Shuffle:** 3001 (Frontend Web UI), 5001 (Backend Webhook Listener), 9200/9300/9600/9650 (OpenSearch).
- **TheHive:** 9000 (Web UI).
- **Cassandra:** 7000-7001, 7199, 9042, 9160.

3. Docker Compose Stack Configuration

- All 6 services (shuffle-opensearch, shuffle-backend, shuffle-frontend, shuffle-orborus, cassandra, thehive) run on a custom `soc-network` bridge.
- **JVM Capping:** `MAX_HEAP_SIZE=1g` (Cassandra), `JAVA_OPTS=-Xms1g -Xmx1g` (TheHive), and `OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m` (Shuffle OpenSearch).

- **Container RAM Limits:** Container memory limits are applied to the Dockerized services such as Shuffle, Shuffle OpenSearch, Cassandra, and TheHive to ensure stable operation within the available 8 GB allocated to the Ubuntu VM.

4. Wazuh Alert to Shuffle Integration (Python Webhook)

- **Path:** `/var/ossec/integrations/custom-shuffle`.
- **File Permissions:** Must be `chmod 750` and owned by `root:wazuh` (previously `root:ossec`).
- **Code Details:** The script captures alerts, parses JSON, and forwards it to Shuffle. Crucially, it uses Python's native `urllib.request` (avoiding missing module errors) and includes an Exponential Backoff Retry loop (`for attempt in range(max_retries):`) to handle temporary SOAR unavailability.
- **Wazuh config:** The `/var/ossec/etc/ossec.conf` file contains an `<integration>` block pointing to the Shuffle backend `http://192.168.196.132:5001/api/v1/hooks/...` matching Level 12 alerts.

5. Detection Engineering & Sigma to Wazuh Mapping

- **T1033 (System Owner/User Discovery):** Command `whoami`. Wazuh Custom Rule `100005` (Level 12) triggered via `sysmon_event1`.
- **T1003.001 (OS Credential Dumping: LSASS):** Wazuh Custom Rule `100010` looking for `sysmon_event10` targeting `(?i)lsass.exe`. Rule `100011` looking for `(?i)comsvcs.dll.*MiniDump`.
- **T1003.002 (SAM Credential Dump):** Wazuh Rule `100015` looking for `(?i)reg.*save.*sam`.
- **T1087.001 (Local Account Discovery):** Wazuh Rule `100016` looking for `(?i)net(\d)?.exe\s+(user|localgroup)`.

6. Shuffle Workflow Configuration

- **Workflow Flow:** Webhook (Port 5001) $\rightarrow$ Shuffle Tools (Regex) $\rightarrow$ AlienVault OTX $\rightarrow$ HTTP POST (TheHive).
- **Regex Node:** Used to sanitize Windows file paths. Replaces `\` with `/` on `$exec.data.win.eventdata.commandLine` to prevent JSON escape character crashes (`\W`, `\z`) in subsequent nodes. Output is saved as `$shuffle_tools_1`.

- **AlienVault OTX Node:** Action set to `GET ipv4 search`. Uses API key. Queries extracted indicators (initially tested with hardcoded botnet IP `185.119.73.161`).
- **TheHive HTTP Node:** Posts raw JSON payload containing `Title` (`$exec.rule.description`), `Summary` (including `$exec.agent.name` and `$shuffle_tools_1`), `Severity`, `Tags` (`["Sysmon", "Wazuh", "Enriched"]`), and `Sourceref` (`$exec.id`).

Section 6: Conflicting Information & Reconciliation

- **Wazuh Deployment Methodology:**
 - *Conflict:* Early project discussions and architectural drafts describe Wazuh as part of a fully containerized deployment using Docker Compose. During implementation, however, issues were encountered with running Wazuh reliably in the containerized environment. As a result, the Docker-based deployment was abandoned.
 - *Resolution:* Wazuh was removed from the Docker Compose stack and installed natively on the Ubuntu host, where it is managed through `systemd`. The remaining SOC pipeline components continue to operate as Docker containers, resulting in a hybrid deployment architecture. This native installation represents the final implementation and should be considered the authoritative deployment model for the project.
- **Shuffle Backend Port:**
 - *Conflict:* Initially referenced as port 5000.
 - *Resolution:* Confirmed to actively operate and listen on port `5001` (`0.0.0.0:5001->5001/tcp`). Webhook integrations in `ossec.conf` must point to 5001.

Section 7: Problems Encountered and Troubleshooting

Issue 1: Storage Exhaustion (Cluster Block Exception)

- **Problem:** OpenSearch crashed with `cluster_block_exception: disk usage exceeded flood-stage watermark, index has read-only-allow-delete block`.
- **Symptoms:** TheHive and Shuffle could not save workflows or generate alerts. `df -h` showed 99% capacity on / partition.
- **Root Cause:** The 38 GB virtual disk filled up due to unchecked Docker container logs and unused images.

- **Solution:**
 1. Executed `docker system prune -f -a`.
 2. Truncated large logs: `sudo sh -c 'truncate -s 0 /var/lib/docker/containers/*/*-json.log'`.
 3. Cleared apt cache: `sudo apt clean`.
 4. Unlocked OpenSearch via API: `curl -X PUT "localhost:9200/_all/_settings" ... -d '{"index.blocks.read_only_allow_delete": null}'`.
 5. *Future Action:* Expanding the virtual disk to 60-80 GB is required.

Issue 2: Webhook JSON Serialization Failure (400 Bad Request)

- **Problem:** Shuffle rejected incoming Wazuh alerts.
- **Symptoms:** Shuffle returned `400 Bad Request: Unrecognized character escape 'z' or \W`.
- **Root Cause:** Windows telemetry generates file paths like `C:\Windows\...` or `C:\Users\z\...`. In JSON, `\` is an escape character. `\W` and `\z` are invalid, crashing the SOAR JSON parser.
- **Solution:** Applied `json.dumps()` in the Wazuh Python webhook script to automatically escape backslashes, and implemented a Shuffle Regex node to swap `\` for `/` before passing data to TheHive.

Issue 3: TheHive Connector App Failures

- **Problem:** TheHive node in Shuffle failed to save or threw JSON token errors (`Unrecognized token '$'`).
- **Symptoms:** Error logs pointing to `flag: ${flag}` and `pap:.`
- **Root Cause:** The graphical Shuffle connector for TheHive passed empty string placeholders (like `${flag}`) directly into the JSON array unquoted, breaking strict JSON parsing.
- **Solution:** Abandoned the visual TheHive App node. Replaced it with a standard HTTP POST node containing a hardcoded, clean JSON payload.

Issue 4: Wazuh Container SSL Hostname Mismatch

- **Problem:** During an early implementation phase, Wazuh was deployed using Docker Compose. This configuration exposed SSL hostname mismatches between the Wazuh Manager and Wazuh Indexer caused by the evaluation certificates bundled with the Docker images.
- **Symptoms:** Logs showed `x509: certificate is valid for demo.indexer, not wazuh.indexer` and `Received fatal alert: bad_certificate`.
- **Root Cause:** The default evaluation TLS certificates baked into the Wazuh Docker images are hardcoded for the hostname `demo.indexer`.
- **Solution:** Although the issue was temporarily mitigated by introducing the `demo.indexer` alias, the containerized deployment was ultimately abandoned in favor of a native installation.

Issue 5: OpenSearch Initialization Permission Denied

- **Problem:** Attempting to run `securityadmin.sh` to initialize the Wazuh Indexer failed. (This issue occurred during the abandoned containerized deployment)
- **Symptoms:** `OCI runtime exec failed: ... permission denied`.
- **Root Cause:** The script lacked executable (`+x`) permissions inside the pre-baked container, and calling it via shell stripped internal Java environment variables.
- **Solution:** Ran `docker exec -it --user root wazuh-indexer bash -c "export JAVA_HOME=/usr/share/wazuh-indexer/jdk && bash /usr/share/wazuh-indexer/plugins/opensearch-security/tools/securityadmin.sh ... -icl"`.

Issue 6: Atomic Red Team Download Failure

- **Problem:** `Invoke-WebRequest` failing to download the ART master zip.
- **Symptoms:** `The connection was closed unexpectedly`.
- **Root Cause:** Default PowerShell sessions drop connections to GitHub due to legacy TLS settings.
- **Solution:** Forced TLS 1.2 using `[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12` and used the `.NET WebClient` class (`$webClient = New-Object System.Net.WebClient`) to bypass `Invoke-WebRequest` instability.

Section 8: Commands, Scripts, and Code Snippets

1. Wazuh Custom Integration Python Script ([/var/ossec/integrations/custom-shuffle](#))

Implemented with exponential backoff and error handling.

```
#!/var/ossec/framework/python/bin/python3
import sys
import json
import urllib.request
import time

# 1. Read alert
with open(sys.argv, 'r') as alert_file:
    alert_data = json.loads(alert_file.read())

webhook_url = sys.argv

# 2. Package data
req = urllib.request.Request(webhook_url)
req.add_header('Content-Type', 'application/json; charset=utf-8')
jsondata = json.dumps(alert_data).encode('utf-8')
req.add_header('Content-Length', len(jsondata))

# 3. Send with Exponential Backoff
max_retries = 3
for attempt in range(max_retries):
    try:
        response = urllib.request.urlopen(req, jsondata, timeout=10)
        break
    except Exception as e:
        if attempt < max_retries - 1:
            time.sleep(2 ** attempt)
```

2. Sysmon Wazuh Agent Configuration ([ossec.conf](#) Windows)

```
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

3. Wazuh Custom Level 12 Rule (local_rules.xml)

```
<group name="custom_sysmon_alerts,">
  <rule id="100016" level="12">
    <if_sid>92031</if_sid>
    <field name="win.system.eventID">^1$</field>
    <field name="win.eventdata.commandLine"
type="pcr2">(?!net(\d)?.exe\s+(user|localgroup)</field>
    <description>SOC Alert: Malicious Local Account Discovery (Atomic Red Team
T1087.001)</description>
    <mitre>
      <id>T1087.001</id>
    </mitre>
  </rule>
</group>
```

4. Raw JSON Override for TheHive Node (Shuffle)

```
{
  "title": "$exec.rule.description",
  "description": "Generated by Wazuh via Shuffle",
  "summary": "#### Wazuh Alert Details\n\n**Agent Name:** $exec.agent.name\n\n**Agent
IP:** $exec.agent.ip\n\n#### Execution Details\n\n**Command Line:**
$shuffle_tools_1\n\n---\n\n#### Threat Intelligence (AlienVault OTX)\n\n**Queried Indicator:**
$exec.agent.ip\n\n**Validation:** Data automatically routed and checked by SOAR.",
  "severity": 4,
  "source": "Wazuh",
  "sourceRef": "$exec.id",
  "type": "external",
  "tlp": 3,
  "pap": 3,
  "flag": true,
  "tags": ["Sysmon", "Wazuh", "Enriched", "CRITICAL_INCIDENT"]
}
```

Section 9: Current State and Operational Assessment

- **Infrastructure:** Operational. The production lab operates using a hybrid deployment model. Wazuh is installed natively on the Ubuntu host, while Shuffle, TheHive, Cassandra, and supporting services are deployed as Docker containers. This architecture reduces complexity, improves stability, and fits within the project's hardware constraints.
- **Endpoint Telemetry:** Operational. Windows 10/11 VM is actively generating Sysmon event logs and forwarding them via the Wazuh Agent.
- **Detection Engine:** Operational. Wazuh is correctly matching custom XML rules and elevating targeted events (like `whoami` and ART executions) to Level 12 critical status.
- **Automation/Orchestration:** Operational. Level 12 alerts are triggering the Python script in Wazuh, pushing JSON to Shuffle, executing Regex path sanitization, querying AlienVault OTX, and issuing HTTP POST requests directly to TheHive.
- **Case Management:** Operational. TheHive is successfully generating Alerts populated with extracted indicators and MITRE tags.

Section 10: Technical Debt & Risks

- **Storage Exhaustion Risk (High):** The 50GB allocated disk size remains extremely tight. Despite the cron pruning script, continuous log ingestion from Sysmon and OpenSearch indices will eventually require expanding the virtual hard drive (to 60-80 GB) and resizing the Linux partition.
- **Fragile TheHive Integration (Debt):** Utilizing a raw HTTP POST block to bypass TheHive's connector UI bugs means that any future API schema changes by TheHive will silently break ticket creation without graphical warnings.
- **Security Posture Limitations (Debt):** `plugins.security.disabled=true` is used in OpenSearch to allow seamless connection between Shuffle and its database. This is inherently insecure and unacceptable for production environments outside of an isolated lab.
- **Memory Swap Wear (Medium):** Hard-capping JVMs and forcing `swappiness=1` heavily mitigates host SSD degradation, but sudden traffic spikes triggering the 4-6GB Linux swap file will still cause high physical drive I/O.

Section 11: Future Work and Roadmap

- **Phase 1: Scale Adversary Emulation:** Execute remaining mapped Atomic Red Team techniques on the Windows Endpoint, specifically Credential Dumping (T1003.001) and Event Log Clearing (T1070.004).
- **Phase 2: Scale Detection Rules:** Draft and compile the custom Wazuh XML rules to catch Sysmon Event IDs 10 (lsass.exe targeting) and 1 (wevtutil.exe cl).
- **Phase 3: Dynamic SLA Routing:** Re-wire Shuffle to use a "Multi-Path Split" (Switch) node. Route webhooks dynamically based on `$exec.rule.id` to assign variable severities (e.g., Critical for T1003.001 vs High for T1070.004) and map accurate SLA timers in TheHive.
- **Phase 4: Establish Continuous Validation:** Schedule cron jobs on the target endpoint to trigger Invoke-AtomicTest on a weekly schedule. Build a Shuffle loop to flag a `#DetectionGap` in TheHive if Wazuh fails to catch the execution within a 5-minute window.
- **Phase 5: Metrics Dashboard:** Implement a reporting system to calculate Mean Time to Detect (MTTD), Mean Time to Resolve (MTTR) per severity tier, and False Positive Rates (FPR).